CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 2 – CO2

Informed Search • Online Search • Game Playing • CSP • Optimisation

Assignment details	Information
Total marks	50
Number of tasks	7
Subject area	Artificial Intelligence
Student name	Abinekavarshini.M
Register number	1924260333
Submission date	_____

Preparation note: This report provides reasoned designs, mathematical analysis, pseudocode and reproducible Python starting points. The assignment specifically requires the student's own measured outputs and raw data. Any table identified as an illustrative prediction is not a claim that an experiment was executed; run the code on your chosen test cases and replace those entries with your actual measurements.

How to use this report

The assignment is assessed on the sequence Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate. For full credit, retain the map definitions, tree structures, random seeds, raw measurements and source code used in your final submission. The explanations below follow the task order and mark distribution in the supplied assignment sheet.

Common measurement definitions

Nodes generated: states created as successors. **Nodes expanded:** states removed from the frontier and whose successors are examined. **Frontier:** discovered but not yet expanded states. **Optimal path:** a path with minimum total edge cost. **Execution time:** report in seconds or milliseconds, but do not use it as the only performance measure.

For fair comparisons, use the same start, goal, obstacles, edge costs, neighbour order and hardware for every algorithm. Count operations in the same way across implementations.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

1.1 Prediction and algorithmic reasoning

Uniform-Cost Search (UCS) expands the frontier state with the smallest accumulated path cost g(n). It is complete and optimal when step costs are positive. It does not use a heuristic, so it may explore many cells that are not directed toward the goal.

Greedy Best-First Search (GBFS) selects the state with the smallest heuristic h(n), an estimate of distance to the goal. It can move quickly toward a nearby-looking goal, but can be misled by walls, detours and dead ends. It is not generally optimal.

A* selects the state with minimum f(n)=g(n)+h(n). With a suitable admissible heuristic (and graph-search handling of repeated states), it is optimal for non-negative costs; with a consistent heuristic it need not reopen closed states.

Map condition	UCS prediction	Greedy prediction	A* prediction
Open grid	Expands a broad cost-ordered region; optimal but may expand many cells.	Usually heads directly toward the goal; often few expansions, but no optimality guarantee.	Usually fewer expansions than UCS while retaining optimality.
Cluttered grid	Finds the least-cost route; explores around obstacles according to g.	May aim at a blocked or misleading direction and take a long detour.	Balances detour cost and goal direction; typically robust and optimal.
Maze-like map	May expand many reachable corridors before reaching the goal.	Can repeatedly choose a corridor that looks closer but is a dead end.	Heuristic helps, but walls can make h weak; still optimal.
Dead-end trap	Eventually backtracks through cost-ordered alternatives.	Can spend effort deep in a misleading branch.	The g term penalises expensive detours; it can recover and remains optimal.

1.2 Admissible heuristic

For a four-directional grid in which each move costs 1, use the Manhattan-distance heuristic: $\mathbf{h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|}$.

This heuristic is admissible because it is the exact shortest distance in a relaxed map where all obstacles are removed. Adding obstacles cannot make the true route shorter than the obstacle-free route. Therefore h(n) never overestimates the actual remaining cost. It is also consistent for unit-cost four-neighbour moves.

If diagonal movement is allowed, the movement model and heuristic must be changed together; Manhattan distance is not appropriate for unrestricted diagonal moves.

1.3 Three deliberately contrasting maps

Map	Design	What it tests
Map A – Open	A 10×10 empty grid; start (0,0), goal (9,9).	Baseline where Manhattan distance closely guides A* and Greedy.
Map B – Barrier/maze	A 10×10 grid with a long wall and one gap near an edge; start and goal on opposite sides.	Tests detours and the effect of obstacles on a straight-line heuristic.
Map C – Dead-end trap	A 9×9 grid with a long open-looking branch toward the goal that terminates, plus a winding valid corridor.	Exposes Greedy's susceptibility to a low h-value in a dead end.

1.4 Implementation outline

Represent each traversable cell as (row, column). Generate up, down, left and right neighbours that remain inside the grid and are not obstacles. Maintain a priority queue and a best-known g-score for each cell. UCS uses priority g; Greedy uses priority h; A* uses priority g+h. Store parent pointers to reconstruct the route.

```
frontier = priority_queue()
frontier.push(start, priority(start))
g[start] = 0
while frontier is not empty:
    current = frontier.pop_min()
    if current == goal:
        return reconstruct_path(parent, goal)
    if current was already expanded with a better cost:
        continue
    mark current expanded
    for next in valid_neighbours(current):
        new_g = g[current] + move_cost(current, next)
        if next not in g or new_g < g[next]:
            g[next] = new_g
            parent[next] = current
            frontier.push(next, priority(next))
return FAILURE
```

Set priority(n)=g(n) for UCS, h(n) for Greedy, and g(n)+h(n) for A*. For UCS and A*, use a priority queue with stale-entry checking. For Greedy, store the best g as well if path reconstruction and duplicate control are required.

1.5 Comparison and recommendation

Use a result table with one row per map and algorithm, recording nodes generated, nodes expanded, maximum frontier size, path cost, optimality and runtime. The expected qualitative result is that Greedy can be fastest on easy open maps but may return a non-optimal route or perform poorly in traps; UCS is reliable but may expand more; A* is the strongest general-purpose choice when the heuristic is informative.

Operational recommendation: Since about 95% of shuttle trips occur on open roads with occasional detours, recommend A* with Manhattan distance for a dependable balance of route quality and search effort. If all moves have equal cost and the team values simplicity over reducing expansions, UCS is a valid optimal baseline. Greedy should not be the default where route optimality and predictable detour handling matter.

The measured recommendation must be checked against the three implemented maps; do not report invented node counts as observed data.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

2.1 Online search agent

An online search agent acts without knowing the complete map. It senses adjacent cells, chooses a move, observes the result and updates its internal map. Online depth-first search (DFS) explores an unknown environment by moving to an unvisited neighbour when possible and backtracking when all known neighbours have been explored.

For a reversible grid, maintain for every visited cell its known neighbours and exploration status. A stack can represent the DFS route. When no unvisited neighbour remains, move to the parent cell. The process stops when the dock is found or all reachable cells have been explored.

2.2 Experimental design

Test factor	Suggested levels	Measurements
Target distance	Near, medium and far targets	Moves to target; ratio to known-map shortest path.
Reachability	Reachable and blocked-off target	Success/failure; total exploration moves.
Maze size	Small, medium and large grids	Moves; visited cells; peak memory.
Dead-end density	Low, medium and high	Revisits; backtracking moves; competitive ratio.
Repeated trials	Same map over 1, 5 and 20 missions	Whether learned information reduces later travel.

The competitive ratio for a successful run is **travel cost made by the online agent ÷ shortest-path cost with full map knowledge**. If the target is unreachable, this ratio is undefined because the optimal target-reaching cost does not exist; report exploration cost and failure separately.

2.3 When is online search reasonable?

Online search is suitable when the map is genuinely unknown or changes frequently, local sensing is available, the environment is small or moderate, and the rover must make progress without waiting for a complete map. It is also useful when a safe exploration policy is more important than finding the globally shortest route immediately.

It becomes unattractive when the environment is very large, dead ends are common, the target is far away, backtracking is expensive, or the target is unreachable. A DFS-based rover can spend substantial time exploring irrelevant regions. If a complete reliable map can be obtained cheaply, an offline planner such as A* may be more efficient.

2.4 Changed condition: 20 rovers, 20 stable facilities

Because the maps remain unchanged for months, the organisation should not repeatedly perform full online exploration as the normal operating mode. Each facility can be mapped once, checked for safety, and stored. Later missions can plan routes on the known map using A* or another suitable offline planner. Online sensing should remain as a safety layer to detect unexpected changes.

LRTA* or another learning approach can improve repeated navigation by updating heuristic estimates from experience, but learning is most valuable when a map cannot simply be shared or centrally stored. With 20 stable facilities, a shared verified map and offline planning are usually the more efficient primary design. Keep online exploration as a fallback for map changes and unmapped sections.

For the report, run each map over multiple trials and present actual move counts, revisited-cell counts and ratios. Separate first-trial exploration from later learned or map-informed missions.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

3.1 Minimax and alpha-beta

Minimax models a two-player, zero-sum, deterministic game. MAX chooses the move with the highest backed-up value; MIN chooses the lowest. Alpha-beta pruning gives the same minimax decision but avoids exploring branches that cannot affect the final choice.

Alpha (α) is the best value MAX can guarantee along the current path. **Beta (β)** is the best value MIN can guarantee. When $\alpha \geq \beta$, the remaining children of that node can be pruned.

3.2 Three tree designs and predictions

Test tree	Design / move order	Prediction
A – Good ordering	At each MAX node, examine the strongest move first; at each MIN node, examine the weakest response first.	Many early cutoffs; alpha-beta visits far fewer nodes than plain minimax.
B – Poor ordering	Place the least useful moves first and the best pruning-causing moves last.	Few or late cutoffs; node count approaches minimax.

Test tree	Design / move order	Prediction
C – Equal leaf values	Use a balanced tree with identical leaf values.	Cutoffs depend on traversal and bounds; equal values can still trigger $\alpha \geq \beta$, so measure rather than assume no pruning.

A third valid condition is random move ordering. Repeat it with several fixed random seeds and report the mean and range of visited nodes and cutoffs.

3.3 Pseudocode

```
MINIMAX(node, depth, maximizing):
    if depth == 0 or node is terminal:
        return evaluate(node)
    if maximizing:
        value = -infinity
        for child in children(node):
            value = max(value, MINIMAX(child, depth-1, False))
        return value
    else:
        value = +infinity
        for child in children(node):
            value = min(value, MINIMAX(child, depth-1, True))
        return value

ALPHA_BETA(node, depth, alpha, beta, maximizing):
    if depth == 0 or node is terminal:
        return evaluate(node)
    if maximizing:
        value = -infinity
        for child in ordered_children(node):
            value = max(value, ALPHA_BETA(child, depth-1,
                                          alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta: break
        return value
    else:
        value = +infinity
        for child in ordered_children(node):
            value = min(value, ALPHA_BETA(child, depth-1,
                                          alpha, beta, True))
            beta = min(beta, value)
            if beta <= alpha: break
        return value
```

3.4 Complexity and interpretation

For branching factor b and depth d, plain minimax examines $O(b^d)$ leaves and $O(b^d)$ total nodes. Alpha-beta has worst-case complexity $O(b^d)$ with poor ordering, but with ideal ordering its effective complexity is approximately $O(b^{d/2})$ in terms of leaf examination. The improvement depends strongly on move ordering.

A tree with the same branching factor and depth can therefore have very different alpha-beta performance. Branching factor and depth alone do not describe the practical work; the order in which promising moves are examined determines how early bounds become strong enough to prune.

Record total recursive calls (nodes visited), leaf evaluations and cutoff count for both algorithms. The minimax value must be identical in every test. If the values differ, check the MAX/MIN alternation, terminal test, child order and alpha/beta updates.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

4.1 Mathematical node count

For a full game tree with branching factor b and lookahead depth d, the number of nodes at level k is b^k. Therefore:

Total nodes including the root = $1 + b + b^2 + \ldots + b^d = (b^{d+1} - 1)/(b - 1)$, for $b \neq 1$.

The number of leaf positions at depth d is $\mathbf{b^d}$. If b=1, the total number of nodes is d+1.

Branching factor b	Depth d	Leaves b^d	Total nodes
3	2	9	13
3	3	27	40
3	4	81	121
4	3	64	85

4.2 Evaluation function

When the search reaches the depth limit before a terminal game state, the evaluation function estimates how favourable that state is for MAX. A simple example for a generic board game is:

Eval(s) = 3 × (MAX material – **MIN material) + 2 × (MAX mobility** – **MIN mobility)**.

The weights are illustrative and must be adapted to the game. Positive values favour MAX; negative values favour MIN. Terminal wins and losses should receive values larger in magnitude than ordinary heuristic scores so that the search prefers a forced win.

4.3 Experimental plan

Use at least three depths, for example d=2, 3 and 4, while keeping the branching factor and evaluation function fixed. Count nodes expanded and evaluation-function calls. In a full uniform tree, the number of depth-limited evaluations is b^d. In a real game, terminal states can occur earlier and legal move counts can vary, so measured values may be lower or irregular.

A simple reproducible synthetic tree is useful for validating the mathematical count. For an actual game, report the board positions, branching factor assumptions and evaluation formula. Compare the measured node count with the theoretical full-tree count.

4.4 Scalability

Define a response-time target before testing, such as 1 second per move on the test computer. Plain minimax becomes impractical when b^d causes the measured runtime to exceed that limit or the node count exceeds the available computational budget. The main source of inefficiency is the exponential growth in possible future move sequences.

A redesigned strategy should reduce the number of states examined without changing the decision where exactness is required, for example by pruning branches using bounds, ordering moves effectively, reusing previously computed states, or limiting search selectively. Task 4 asks for identification and design requirements, not implementation of the redesign.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

5.1 CSP representation

Represent each exam as a variable X■. Its domain D■ contains available time slots. If two exams share at least one student, add a binary constraint X■ ≠ X■. The CSP consists of variables, domains and constraints.

A value v in D■ has support in neighbouring variable X■ if there is at least one value w in D■ that satisfies the constraint between X■ and X■. If no such w exists, v can be removed.

5.2 Arc-consistency propagation

The given principle is the basis of arc consistency (commonly implemented using the AC-3 work-queue method). For every directed arc (X■, X■), revise D■ by removing values that have no compatible support in D■. If D■ changes, neighbouring arcs pointing into X■ must be reconsidered.

```
AC_PROPAGATE(domains, neighbours):
    queue = all directed arcs (Xi, Xj)
    while queue is not empty:
        (Xi, Xj) = remove_one(queue)
        if REVISE(Xi, Xj):
            if domain[Xi] is empty:
```

```
                return FAILURE
            for Xk in neighbours[Xi], where Xk != Xj:
                add (Xk, Xi) to queue
    return SUCCESS

REVISE(Xi, Xj):
    changed = False
    for value in copy(domain[Xi]):
        if no value in domain[Xj] satisfies constraint(Xi=value, Xj):
            remove value from domain[Xi]
            changed = True
    return changed
```

5.3 Backtracking with propagation

Propagation is applied after each tentative assignment. If any domain becomes empty, the branch is inconsistent and backtracking is required. If all variables are assigned consistently, a solution has been found.

```
BACKTRACK(assignment, domains):
    if every variable is assigned:
        return assignment
    choose an unassigned variable Xi
    for value in ordered_values(domains[Xi]):
        if value is consistent with assignment:
            make a copy of domains
            assign Xi = value
            restrict domain[Xi] to {value}
            if AC_PROPAGATE(domains, neighbours) succeeds:
                result = BACKTRACK(assignment, domains)
                if result is not FAILURE: return result
    return FAILURE
```

5.4 Test instances

Instance	Variables and domains	Expected propagation
A – propagation failure	A={1}, B={1,2}, C={2}; constraints A≠B and B≠C.	A=1 removes 1 from B, so B=2; then C cannot be 2. Propagation detects failure without branching.
B – challenge instance	A={1,2}, B={1,2}, C={1,2}; constraints A≠B, B≠C, C≠A.	Arc consistency alone leaves all domains unchanged, but search discovers that a 2-colouring of a triangle is impossible.

For a propagation-only satisfiable example, use a chain A={1}, B={1,2}, C={1,2,3} with constraints A≠B and B≠C: propagation forces B=2, then C can be 1 or 3, so the instance remains satisfiable without search.

5.5 Why propagation is sound

Suppose value v is removed from D■ because a neighbouring variable X■ has no value compatible with v. In every complete solution, X■=v would have to be paired with some allowed value of X■ satisfying their constraint. No such value exists, so no valid complete solution can contain X■=v. Removing v therefore cannot remove a genuine solution.

Compare plain backtracking and backtracking with propagation on nodes visited, backtracks, constraint checks and domain wipe-outs. As the number of exams increases, the search space can grow exponentially, while propagation can reduce domains early. Propagation has a cost of its own, so record constraint checks as well as search nodes.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

6.1 Exhaustive-search design

This is the travelling salesperson problem (TSP). Fix the depot as the first and last city. Generate every permutation of the n customer locations, calculate the total tour distance including the return to the depot, and retain the least-cost tour.

For n customers, the number of directed tours with a fixed depot is n!. If distances are symmetric, a tour and its reverse have equal cost, so only n!/2 distinct cycles need be checked for n≥2 if reversal duplicates are deliberately eliminated.

```
best_cost = infinity
best_tour = None
for order in all_permutations(customers):
    tour = [depot] + list(order) + [depot]
```

```
    cost = sum(distance(tour[i], tour[i+1])
               for i in range(len(tour)-1))
    if cost < best_cost:
        best_cost = cost
        best_tour = tour
return best_tour, best_cost
```

6.2 Growth prediction

Customers n	Directed tours n!	Approximate implication
4	24	Small enough for direct verification.
6	720	Still manageable.
8	40,320	Noticeably more work.
10	3,628,800	Millions of tours; expensive in a high-level implementation.
12	479,001,600	Usually unsuitable for repeated real-time enumeration.

The factorial growth occurs because there are n choices for the first customer, n–1 for the second, and so on, giving n×(n–1)×…×1=n!. Adding a small number of customers multiplies the search space by the next integer; this is why exhaustive search rapidly becomes impractical.

6.3 Local-search alternative

A practical local-search method is 2-opt. Start with a valid tour. Select two non-adjacent edges and reconnect the tour by reversing the segment between them. If the new tour is shorter, accept it. Continue until no improving 2-opt move is found. Random restarts reduce dependence on one initial tour.

Another option is simulated annealing, which sometimes accepts a worse move early in the search to escape local minima. It gradually reduces the probability of accepting worse moves as the temperature decreases.

6.4 Quality measurement

For each test size where the exhaustive optimum is known, run the local method at least 20 times using different recorded random seeds. For each run calculate: **Percentage gap = 100 × (local cost** – **optimal cost) / optimal cost.**

Report mean, minimum and maximum gap, and mean runtime. A gap of 0% means the local method found an optimal tour. Do not claim local search is optimal in general.

6.5 Why local search can fail

A local minimum is a tour for which no allowed neighbouring move improves the cost, even though a better tour exists elsewhere. A plateau is a region of neighbouring tours with equal cost, so the search may make no progress or wander. Random restarts, larger neighbourhoods or simulated annealing can reduce these problems but do not guarantee the global optimum.

6.6 Deployment decision

Choose a response-time budget appropriate to the delivery system, for example 2 seconds for a planning request. Run exhaustive search at increasing n and select the largest n that consistently finishes within the budget with acceptable memory use. For larger instances, switch to 2-opt with restarts or simulated annealing. The switch point must come from measured runtime and operation counts on the target hardware, not from an unsupported universal number.

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: plain minimax search from Task 4. This is a suitable redesign case because its exponential growth is clear and measurable.

A. Diagnose the computational bottleneck (2 marks)

The main bottleneck is repeated evaluation of a large game tree. With branching factor b and depth d, the full tree contains approximately b^d leaves. Most processing time is spent generating successor states and recursively evaluating positions. If the same game state can be reached through different move sequences, the same future position may also

be evaluated repeatedly.

B. Define the design objective (2 marks)

Objective: return the same minimax value as the original depth-limited search while reducing the number of evaluated nodes on representative test trees. A measurable target is to reduce leaf evaluations by at least 30% on a fixed test suite, with no change in the selected minimax value. This is a target to test, not a guaranteed result.

C. Revised strategy (4 marks)

Use alpha-beta pruning with move ordering. Before the recursive search, order moves so that likely strong moves for the current player are considered first. Good early moves establish a strong alpha or beta bound, allowing later branches that cannot influence the final decision to be skipped. For repeated states, an optional transposition table can store previously computed values and reuse them.

Trade-offs: move ordering requires an estimate or a small amount of extra work. A transposition table consumes memory and must store depth and bound type correctly. Poor ordering may provide little benefit. The minimax result remains unchanged when alpha-beta is implemented correctly.

```
ORDERED_ALPHA_BETA(state, depth, alpha, beta, player):
    if terminal(state) or depth == 0:
        return evaluate(state)
    moves = legal_moves(state)
    sort moves by estimated promise for player
    if player == MAX:
        value = -infinity
        for move in moves:
            value = max(value, ORDERED_ALPHA_BETA(
                apply(state, move), depth-1, alpha, beta, MIN))
            alpha = max(alpha, value)
            if alpha >= beta: break
        return value
    else:
        value = +infinity
        for move in moves:
            value = min(value, ORDERED_ALPHA_BETA(
                apply(state, move), depth-1, alpha, beta, MAX))
            beta = min(beta, value)
            if beta <= alpha: break
        return value
```

D. Predict before implementing (1 mark)

Prediction: on trees where move ordering places strong moves first, alpha-beta will visit fewer nodes and call the evaluation function fewer times than plain minimax, while returning the same root value. On badly ordered trees, the improvement may be small.

E. Implement and validate (2 marks)

Run plain minimax and the redesigned version on identical test trees, with the same leaf values and depth. Record the root value, nodes visited, leaf evaluations, cutoffs and runtime. Repeat on good, poor and random move orders. The root values must match exactly.

Measure	Original minimax	Redesigned search
Root minimax value	Record actual result	Must equal original
Nodes visited	Record actual count	Expected to decrease with useful ordering
Leaf evaluations	Record actual count	Expected to decrease with cutoffs
Cutoffs	0	Record actual number
Runtime	Measure on same machine	Measure on same machine

F. Defend the final design (2 marks)

The redesign succeeds if it returns the same minimax value while reducing measured node visits or leaf evaluations on the test cases. Report the percentage reduction as 100×(original count−new count)/original count. Include the extra cost of ordering and memory if a transposition table is used.

The original minimax may still be preferable for very small trees, where ordering overhead is not worthwhile, or where the tree structure offers no useful cutoffs. A redesign that fails to improve a particular test should be analysed honestly: poor move ordering, equal values, a shallow tree or overhead may explain the result.

Overall conclusion

No single AI search method is best under every condition. UCS guarantees optimality without a heuristic; Greedy can be fast but may be misled; A* balances path cost and heuristic guidance. Online search is useful in unknown or changing environments, but stable repeated environments favour mapping and offline planning. Alpha-beta preserves minimax decisions while reducing work when move ordering is effective. Constraint propagation removes unsupported domain values before or during backtracking. Exhaustive TSP search proves optimality for small cases but factorial growth requires local search for larger cases. A sound redesign must be justified by measured operation counts and must state its trade-offs.

Final submission checklist

• Include the exact maps, game trees, CSP instances and TSP distance matrices used.

• Attach runnable Python source code and note the Python version and hardware.

• Record raw measurements, including operation counts and fixed random seeds.

• Replace all illustrative predictions or blank measurement cells with your own experimental results.

• Include graphs where useful and ensure every conclusion refers to evidence.

• Be prepared to explain each algorithm, design choice and result during the viva.

Source basis

Prepared from the uploaded document *CSA17 – Artificial Intelligence, Assignment 2 – CO2* (7 pages, 50 marks). Explanations use standard AI search and optimisation principles to answer the tasks specified in that document.